웹 모의해킹 보고서

KnockOn Bootcamp 2기 – Web

박승한

목차

1. 개요 3
1.1 점검 기간 3
1.2 점검자 3
1.3 점검 대상 3
1.4 점검 목적 3
1.5 점검 방법 3
2. 취약점 요약 4
3. 취약점 분석 5
3.1 SQL 삽입 (SQL Injetion) 5
3.2 교차 사이트 스크립팅 (Cross-Site Scripting) 7
3.3 파일 업로드 (File Upload) 10
3.4 설정 (Configuration) 13
3.5 경로 탐색 및 로컬 파일 포함 (Path Traversal & Local File Inclsion) 15
3.6 소스코드 내 중요 정보 노출 17

1. 개요

1.1 점검 기간

시작일: 2024. 9. 19.

종료일: 2024. 9. 30.

1.2 점검자

박승한

1.3 점검 대상

PHP로 개발된 게시판 웹사이트

1.4 점검 목적

PHP로 개발된 웹사이트의 보안 취약점을 식별하고, 이를 패치하여 웹사이트의 안전한 운영을 지원하는 것을 목적으로 한다.

1.5 점검 방법

1) **화이트 박스 테스트**: 사이트의 소스코드를 분석하여 취약점을 식별함. 사용자와 데이터베이스 간의 상호작용 및 입력 검증 등을 확인.

2) **블랙 박스 테스트**: 게시판을 이용하는 사용자의 입장에서 웹사이트에 대한 외부 공격을 테스트함. SQL Injection, XSS, 파일 업로드 등을 테스트.

2. 취약점 요약

번호	취약점 이름	약어	발견
1	SQL 삽입	SQL Injection	O
2	교차 사이트 스크립팅	XSS (Cross-Site Scripting)	O
3	파일 업로드	File Upload	O
4	설정	Configuration	O
5	디렉토리 나열	Directory Listing	O
6	경로 탐색	Pass Traversal	O
7	로컬 파일 포함	LFI (Local File Inclusion)	O
8	소스코드 내 중요 정보 노출		O

3. 취약점 분석

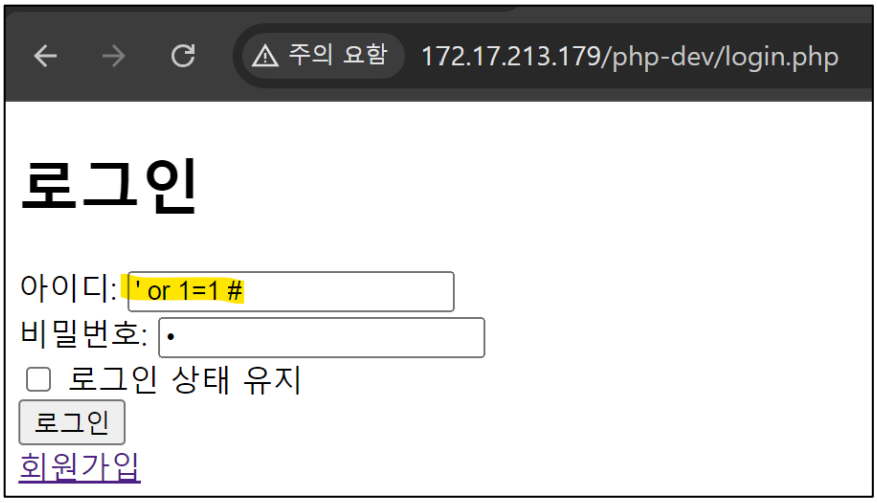
3.1 SQL 삽입 (SQL Injetion)

1) 문제점

SQL 삽입은 웹 애플리케이션에서 사용자 입력을 적절하게 검증하지 않았을 때 발생하는 취약점이다. 공격자가 서버의 데이터베이스 쿼리를 조작할 수 있게 되어 정보 유출 등의 문제가 발생한다.

2) 취약점 발생 위치

/login.php



[그림 1] 공격 쿼리 입력

[그림 2] SQL Injection 성공

3) 대응 방안

- 준비된 문장(Prepared Statements)을 사용한다.
- 입력 값 검증 및 필터링을 수행한다.

4) 패치 적용

login.php

```
$query = "SELECT id, password FROM users WHERE username = '$username' AND password =
'$password'";
$result = $mysqli->query($query);
```

[패치 전] 사용자 입력을 직접 쿼리에 사용

```
$stmt = $mysqli->prepare("SELECT id, password FROM users WHERE username = ?");
$stmt->bind_param("s", $username);
$stmt->execute();
$stmt->fetch();
$stmt->close();
```

[패치 후] prepare 함수를 적용하여 입력 값을 SQL 쿼리가 아닌 단순 데이터로 처리

3.2 교차 사이트 스크립팅 (Cross-Site Scripting)

1) 문제점

교차 사이트 스크립팅은 공격자가 악성 스크립트를 웹 페이지에 삽입할 수 있게 하는 취약점이다. 입력값 검증이 없거나, 페이지 출력 시 적절한 인코딩을 수행하지 않을 때 발생한다. 사용자가 해당 웹 페이지에 접근 시, 악성 스크립트가 실행되어 쿠키, 세션 등의 정보가 탈취될 수 있다.

2) 취약점 발생 위치

/create_post	/update_post	/read_post

/create_post와 /update_post에서 작성한 악성 스크립트를 /read_post에서 출력 시 발생

[그림 3] 게시물에 악성 스크립트 삽입

[그림 4] 악성 스크립트가 포함된 게시물

[그림 5] 게시물에 접근 시 악성 스크립트 실행

3) 대응 방안

- 데이터 출력 시 HTML 인코딩을 적용한다.
- 스크립트가 포함되지 않게 사용자 입력 검증을 수행한다.
- 컨텐츠 보안 정책 (Content Security Policy)을 설정하여 스크립트 실행 제어 및 외부 스크립트 로딩을 제한한다.

4) 패치 적용

read_post.php

```
<p><?php echo nl2br($post['content']); ?></p>
```

[패치 전] $post['content']를 그대로 출력

```
<p> echo nl2br(htmlspecialchars($post['content'])); </p>
```

[패치 후] HTML 인코딩을 하는 htmlspecialchars를 적용하여 스크립트가 실행되지 않도록 함

3.3 파일 업로드 (File Upload)

1) 문제점

파일 업로드는 공격자가 악성 코드가 포함된 파일을 서버에 업로드할 수 있도록 허용 하는 취약점이다. 업로드된 파일을 아용해 공격자가 서버에서 원격으로 코드를 실행하는 RCE(Remote Code Execution)공격으로 이어질 수 있다.

2) 취약점 발생 위치

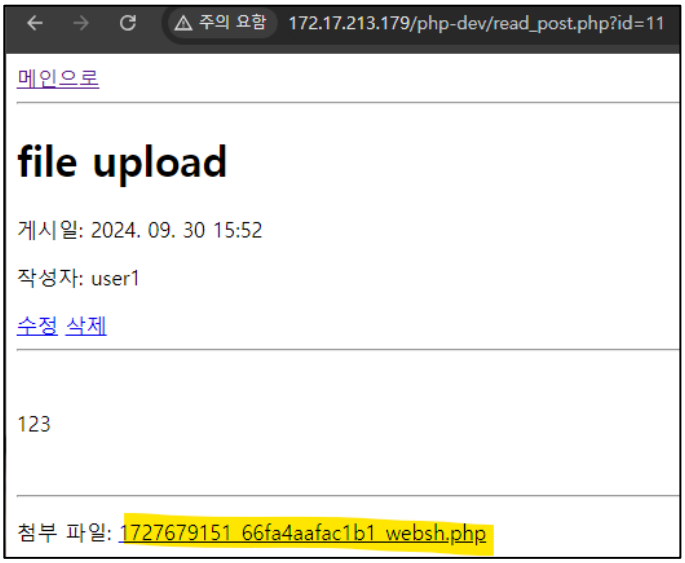
/create_post	/update_post

/create_post와 /update_post에서 악성코드가 포함된 파일을 업로드할 수 있음

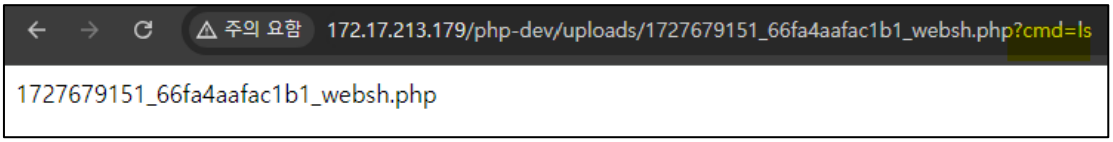
```php
<?php
    system($_GET['cmd'])
?>
```

<websh.php> 원격 코드 실행(RCE)을 가능하게 하는 악성코드 파일

[그림 6] 악성 코드가 포함된 파일 업로드



[그림 7] 성공적으로 게시물이 생성됨



[그림 8] 원격 코드 실행 공격

3) 대응 방안

- 파일 업로드 시 파일 확장자에 대한 검증을 수행한다.
- 업로드된 파일의 실행 권한을 차단한다.
- 파일이 업로드 되는 위치가 노출되지 않게 한다.

4) 패치 적용

config.php	functions.php – handleFileUpload()

```
if ($_FILES['file']['error'] === UPLOAD_ERR_OK) {
            $file_ext = strtolower(pathinfo($_FILES['file']['name'], PATHINFO_EXTENSION));
            $max_file_size = MAX_FILE_SIZE; // 5MB

            if ($_FILES['file']['size'] > $max_file_size) {
                $error_message = "파일 용량 초과 (최대 5MB)";
```

[패치 전] 파일 업로드 시 확장자 검증이 없음

```
define('ALLOWED_EXTS', ['jpg', 'jpeg', 'png', 'gif', 'pdf']);
```

```
if ($_FILES['file']['error'] === UPLOAD_ERR_OK) {
            $allowed_exts = ALLOWED_EXTS;
            $file_ext = strtolower(pathinfo($_FILES['file']['name'], PATHINFO_EXTENSION));
            $max_file_size = MAX_FILE_SIZE; // 5MB

            if ($_FILES['file']['size'] > $max_file_size) {
                $error_message = "파일 용량 초과 (최대 5MB)";
            } elseif (!in_array($file_ext, $allowed_exts)) {
                $error_message = "허용되지 않는 파일 형식입니다.";
```

[패치 후] 파일 업로드 시 화이트리스트를 기반으로 파일 검증을 수행

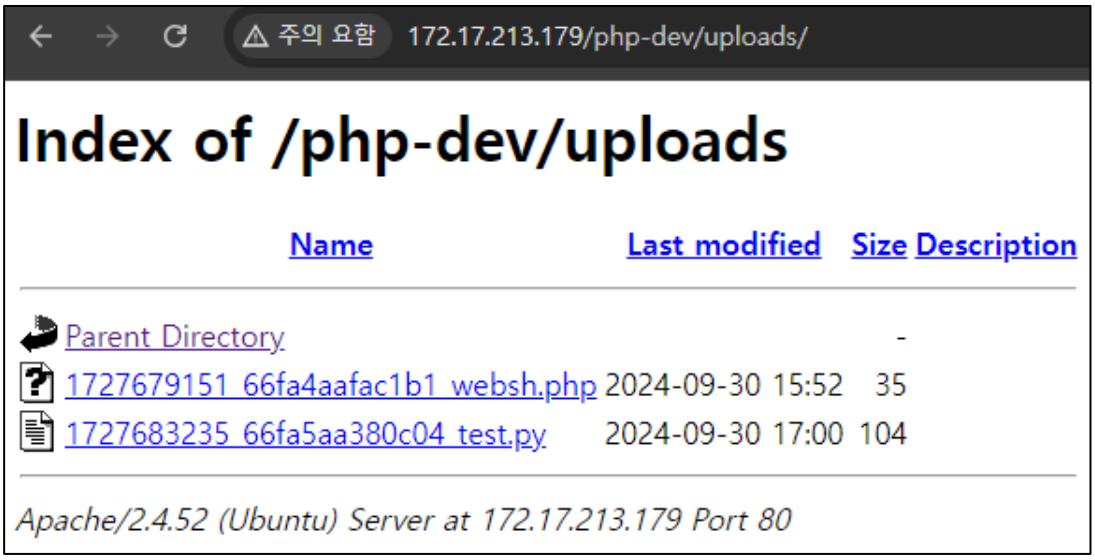
3.4 설정 (Configuration)

1) 문제점

설정 취약점은 웹 서버, 파일 시스템 등의 설정 오류로 발생하는 문제이다. 디렉토리 나열(Directory Listing)로 서버에 존재하는 파일 정보가 유출되면, 공격자가 이를 이용해 원격 코드 실행(RCE) 공격으로도 이어질 수 있다.

2) 취약점 발생 위치

/uploads



[그림 9] 디렉토리 나열

[그림 10] 원격 코드 실행 공격

3) 대응 방안

- 파일이 업로드된 위치로의 접근을 차단한다.

4) 패치 적용

apache2.conf - 아파치 설정 파일

```
<Directory “/var/www/html/php-dev/uploads”>
    Require all denied
</Directory>
```

[패치 후] /uploads 경로로의 접근 차단 코드 추가

3.5 경로 탐색 및 로컬 파일 포함 (Path Traversal & Local File Inclsion)

1) 문제점

경로 탐색(Path Traversal)은 상대 경로를 조작하여 시스템 파일에 접근하게 하는 취약점이다. 로컬 파일 포함(Local File Inclsion)은 사용자로부터 파일 경로를 입력받아 파일을 포함할 때, 경로를 조작하여 서버(로컬)에 있는 다른 파일을 포함하도록 하는 취약점이다. 경로 탐색(Path Traversal)을 통한 로컬 파일 포함 공격이 가능해질 수 있어 /etc/passwd, config.php와 같은 민감한 파일이 유출될 수 있다.

2) 취약점 발생 위치

read_post.php – file 파라미터

[그림 11] 경로 탐색 및 로컬 파일 포함 (1)

[그림 12] 경로 탐색 및 로컬 파일 포함 (2)

3) 대응 방안

- 외부 입력값을 직접 전달하지 못하게 한다.
- 입력값 검증을 수행하여 시스템 파일로의 접근을 제한한다.

4) 패치 적용

functions.php – getPostById()

```
if (isset($_GET['file'])) {
    $file_path = $_GET['file'];
}
```

[패치 전] file 파라미터에 사용자 입력을 포함하는 코드

[패치 후] file 파라미터에 사용자 입력을 포함하는 코드 제거

3.6 소스코드 내 중요 정보 노출

1) 문제점

소스코드 내에 관리자나 데이터베이스의 비밀번호, API키 등 중요한 정보가 포함되어 있는 경우 발생한다.

2) 취약점 발생 위치

login.php

[그림 13] 데이터베이스 정보 유출

3) 대응 방안

- 환경 변수를 통해 데이터베이스 정보를 가져오도록 한다.

4) 패치 적용

config.php	login.php

```
$db_host = 'localhost';
$db_user = 'guest';
$db_password = 'guest';
$db_name = 'board';

echo "<!-- db_host = $db_host -->";
echo "<!-- db_user = $db_user -->";
echo "<!-- db_password = $db_password -->";
echo "<!-- db_name = $db_name -->";
```

[패치 전] login.php에 포함되어 있는 데이터베이스 정보

```
define('DB_SERVER', 'localhost');
define('DB_USERNAME', 'guest');
define('DB_PASSWORD', 'guest');
define('DB_NAME', 'board');
```

[패치 후] login.php에 있는 코드 제거, config.php파일에 환경 변수 설정